

プログラミング応用（知能処理）レポート

36714029 / shiwasu（グループ番号 6）

2026 年 6 月 22 日

表 1: 相互評価

氏名	作業割合 (%)
山本賢人	70
遠藤裕人 (自分)	18
堀浩介	12

作業全体を 100 としたときの、各メンバの作業量を申告すること。
自分の主観を正直に報告すること。グループ内で調整してはならない。

全体的な自己評価／作業時間: A 14 時間

全体的な考察: 「乱択相手に勝てること」と「強い相手に勝てること」は別物である、という点が本演習最大の学びである。課題 1e で機動力を過信した初期 ϕ は対 Random 99% でも対改良版で 30% まで落ちたが、位置評価を主軸に据え機動力・石数を段階別の重みへ修正したところ対改良版 61% と逆転でき、変更前より強くできた（表 7）。この段階別評価の発想を踏まえ、大会版 p26x42 では評価項をさらに増やして序盤・終盤で重み分けし、終盤のみ確定石差を重くすることで安定させ、ベースラインに大幅な改善（表 2）を得た。また高速化では、book により総探索ノードを 39%・総思考時間を 17.7% 削減したが、1 秒あたりの訪問ノード数（nps）はむしろ低下した（表 3）。これは「探索しない最適化」を入れた結果であり、nps だけで速度を測る危うさを実測で理解できた。

全体的な感想: 評価関数や定石をいじるたびに勝率が目に見えて変わるのが面白く、つい対戦回数を増やして時間を使ってしまった。強くするには直感だけでなく、必ず多数対局とプロファイルで検証することが不可欠だと痛感した。

1 概要

1. プレイヤ名: p26x42 (グループ 6) shiwasu
2. 大会成績: 第 1 位 891 pts, 勝率 88.7
3. グループ内での作業分担状況の概要: 課題 1 は分担せずにそれぞれが全てのプログラムを作成した。最終課題では山本が探索アルゴリズムと評価関数、book データの確保など全体的に担当、遠藤 (自分) が課題 1 のテスト・検証やスライド作成、発表、デバック、敵対的オセロ AI の作成を担当、堀がテスト・検証やデバック、敵対的オセロ AI の作成を担当した。
4. 自分の担当範囲と簡潔なアピール: テスト・検証を担当。段階的にベースライン (位置評価のみ) → 段階別動的評価 (p26x42base) → book 版 (p26x42) と多数対局で比較し、ベース AI 比 100%・平均 26.33 石差, book 版でノード 39%削減を確認した。作成したプログラムのコードを解析し、要点を理解してスライドに落とし込んだ。

2 プレイヤについて

大会提出プレイヤ p26x42 について述べる (課題 2c の必須 3 点に対応)。

2.1 使用した探索アルゴリズムとその工夫点

探索は α - β 法を用いる。工夫点は、白番のとき盤面を `flipped()` で反転して常に黒視点で探索する点である (リスト 1)。理由は、評価関数が黒視点で書かれているため、反転すれば「反転後の黒 (=元の白)」の優劣を同じコードで評価でき、探索本体から色の分岐を完全に排除できるからである。打ち切り判定は `depth > depthLimit` で、深さ 0 から着手するため実際の先読み手数は `depthLimit+1` プライになる。さらに序盤は定石 (book) を引いて探索自体を省略し、序盤の手を安定させた。

ソースコード 1: 黒視点固定のための盤面反転 (OurPlayer#think)

```
1 // 黒視点で探索するため、白番のときは盤面を反転
2 MyBoard searchBoard = isBlack() ? this.board.clone() : this.board.
  flipped();
3 this.move = null;
4 maxSearch(searchBoard, NEGATIVE_INFINITY, POSITIVE_INFINITY, 0);
5 this.move = this.move.colored(getColor()); // 反転して読んだ手の色を戻す
```

2.2 評価関数とその工夫点

方針は段階別の動的評価である。ベースオセロ AI がほぼ位置の静的評価しかしていないのに対し、p26x42base は局面を前半・後半に分け、前半は「着手可能数 (機動力)・差

確定石差・フロンティア差」を重み付けして評価し、後半は「着手可能数」を最も重く評価する。機動力で相手の打てる手を狭め、差確定石差で取り返されない優位を積み、フロンティア（空きに接する自石）を減らして相手に手を与えないことを狙う。これがベースラインに対する勝率改善（表2）に寄与した。課題1eで機動力を過信した初期φは対強相手に失敗したが、位置評価を主軸に据えて段階別に重み分けしたところ変更前の改良マス重みに勝ち越せた（表7）。この段階別評価をさらに発展させ、評価項を増やして重み分けし安定させた点が工夫である。

表 2: ベースラインに対する勝率の改善幅（各6局, 平均石差つき）

マッチアップ	W-D-L	勝率	平均石差
ベースオセロ AI vs Random	6-0-0	100.00%	23.17
p26x42base vs Random	6-0-0	100.00%	32.33
p26x42base vs ベースオセロ AI	6-0-0	100.00%	26.33
p26x42 vs p26x42base	5-0-1	83.33%	3.33
p26x42 vs ベースオセロ AI	6-0-0	100.00%	23.67

2.3 実行速度（1秒あたりの訪問ノード数）

cse 本番環境（cse.cs.nitech.ac.jp）で標準盤を初手から終局まで1局実行し、総探索ノード数を総思考時間で割って計測した（表3）。book無効で**9.20 Mnps**、book有効で6.82 Mnpsであった。なおbook有効時にnpsが下がる理由は3.3節で考察する。

表 3: 探索速度（cse 本番, 標準盤 初手～終局 1局）

	総ノード数	総思考時間	探索速度
p26x42 book なし	534,944,374	58.141 s	9.20 Mnps
p26x42 book あり	326,263,489	47.844 s	6.82 Mnps

2.4 その他の工夫点

変形盤面（BLOCKあり）に対応した。MyBoardは反転線の探索でBLOCKを壁として扱い（outflanked）、標準盤・変形盤の両方でリーグ戦が完走することを確認した。また終局時に片方の石が全滅した場合はscore()で空きマスに勝者に加算し、完全勝ちを正しく評価している。

3 考察

3.1 実績に期待通りの好影響を与えた工夫点

段階別の動的評価と定石（book）は期待通り有効だった。位置の静的評価のみのベースオセロAIに対し、p26x42baseは直接対決で6戦全勝・平均26.33石差の圧勝（表2）。機

動力・差確定石差・フロンティア差を段階別に重み付けしたことで、相手の打てる手を狭めつつ取り返されない優位を積めたためである。book を足した p26x42 も p26x42base に 5 勝 0 分 1 敗 (83.33%) と勝ち越し、序盤の手が安定して改善が積み上がったことを確認できた。課題 1 でも探索深さの増加は勝率を単調に押し上げた (表 4)。

3.2 実績に期待ほどの好影響を与えられなかった工夫点

book による勝率の上乗せは限定的だった。p26x42 対ベースオセロ AI の平均石差は 23.67 で、book 無しの p26x42base 対ベース AI (26.33) よりむしろ小さい (表 2)。book の価値は「弱い相手への石差拡大」ではなく「序盤の安定と探索コスト削減」にあり、弱い相手相手では勝敗自体は元々ほぼ決まっていた上乗せ余地が小さかったためと考える。真価は強豪相手の序盤事故防止で出ると見込む。

3.3 実績に期待外れの悪影響を及ぼした工夫点

book 有効時の nps 低下が一見の期待に反した (表 3, 9.20 → 6.82 Mnps)。ただしこれは悪影響ではなく**解釈の問題**である。book は「分岐が広く軽い」序盤ノードを探索せず省くため、実際に探索するのは中終盤の重い局面に偏り、1 ノードあたりの平均コストが上がって見かけの nps が下がる。総ノードは 39%・総思考時間は 17.7%減っており目的は達成している。nps は速度の代理指標にすぎず、「探索しない最適化」の効果は nps では測れないと学んだ。なお課題 1e では初期 ϕ が機動力の過大評価で対強相手 30%まで落ちたが、位置評価主軸+段階別の重み分けへ修正して対改良版 61%へ逆転した (表 7)。大会版もこの段階別評価を踏襲している。

3.4 今後の課題

自分の有効盤面だけでなく負けに至る盤面も book 化して学習させることで、序盤の事故をさらに減らせると考える。WLD 証明は残り石差 25 手の時点で行うのではなく、序盤から行なって精度を上げていくことで負ける場合でもできるだけ取られる石数を減らすことができると思う。

4 課題

4.1 課題 1

課題 1a (コード読解と実験) リスト 1~9 を通読した。実験 1 (探索深さ) と実験 2 (手順並び替え) の結果を表 4・表 5 に示す (各 50 局, 先後 25 局ずつ, vs RandomPlayer)。

プログラム: 探索打ち切りは `depth > depthLimit` (`>=` でない) で行う。深さ 0 から着手するため先読みは `depthLimit+1` プライになる。手順並び替え `order()` は同評価値の手の選択順だけを `Collections.shuffle` でランダム化する (リスト 2)。

ソースコード 2: 探索打ち切りと手順並び替え (MyPlayer)

```

1 boolean isTerminal(Board b, int depth) {
2     return b.isEnd() || depth > this.depthLimit; //
        depth0 から着手→先読みは depthLimit+1 プライ
3 }
4 List<Move> order(List<Move> moves) {
5     List<Move> shuffled = new ArrayList<>(moves);
6     Collections.shuffle(shuffled); // 同評価値手の選択順だけをランダム化
7     return shuffled;
8 }

```

実行例: 盤面描画を省いた自作ベンチ p26x29.Bench (先後を半々で入替) で測定した.

ソースコード 3: 実験 1・実験 2 の実行例

```

1 $ java p26x29.Bench depth
2 ## 実験 1: 探索深さ depthLimit と勝率 (vs RandomPlayer, 各 50局)
3 depthLimit=0 : 39勝 7敗 4分 / 50局 勝率 78.0% 得点率 82.0%
4 depthLimit=1 : 44勝 4敗 2分 / 50局 勝率 88.0% 得点率 90.0%
5 depthLimit=2 : 45勝 3敗 2分 / 50局 勝率 90.0% 得点率 92.0%
6 depthLimit=3 : 49勝 1敗 0分 / 50局 勝率 98.0% 得点率 98.0%
7 depthLimit=4 : 50勝 0敗 0分 / 50局 勝率 100.0% 得点率 100.0%
8 $ java p26x29.Bench shuffle
9 shuffle 有 : 45勝 3敗 2分 / 50局 勝率 90.0% 得点率 92.0%
10 shuffle 無 : 48勝 2敗 0分 / 50局 勝率 96.0% 得点率 96.0%

```

評価: 実験 1・実験 2 の結果を表 4・表 5 にまとめる.

表 4: 実験 1: depthLimit と勝率 (vs Random, 各 50 局)

depthLimit	勝/敗/分	勝率
0	39/7/4	78.0%
1	44/4/2	88.0%
2	45/3/2	90.0%
3	49/1/0	98.0%
4	50/0/0	100.0%

表 5: 実験 2: order() の shuffle 有無 (depth=2, vs Random, 各 50 局)

	勝/敗/分	勝率
shuffle 有	45/3/2	90.0%
shuffle 無	48/2/0	96.0%

実験 1 の考察: 「深く読むほど強い」という当初の見込み通り**成功**した. 勝率は深さに対し単調増加し (78 → 88 → 90 → 98 → 100%), depth0 (評価 1 段) でも 78%あることからマス重み評価自体が乱択より十分強い. depth3 以降はほぼ取りこぼしが無く, 対 Random ではこれ以上の深化の効果は飽和する. 計算時間は指数的に増えるため, 相手の強さに応じて深さを選ぶべきだと確認できた.

実験 2 の考察: shuffle は同評価値の手の選択順だけを変える。ルートの更新条件が `childValue > alpha` のため、同点の手が複数あると先に試した手が採用され、shuffle 無では `findLegalMoves` の盤面走査順（実質固定手）、shuffle 有ではランダムな 1 手を選ぶ。探索の最善値自体は変わらず、勝敗を左右するのは同点時の手だけなので、対 Random では両者とも 90% 超になる。差 6pt (45/3/2 対 48/2/0) は各 50 局の標準誤差（約 4pt）に収まり統計的に有意でなく、shuffle 無が高いのは偶然の範囲だと結論した。shuffle は勝率向上ではなく、手を決定論的にせず相手に読まれにくくする点だと思った

実験 3（読解で生じた疑問 3 つと答え）：(1) **なぜ白番で盤を反転するか**→対称性の利用で探索から色分岐を排除するため。(2) **なぜ最善手を副作用 `this.move` で持ち回るか**→`unit0` の α - β とシグネチャを揃える簡略化。ルートで α 更新時のみ保存し、全枝同点に備え先頭手を仮登録する。(3) **`depthLimit` の境界の意味**→深さ 0 から着手するため先読みは `depthLimit+1` プライになる。

課題 1b (HumanPlayer) プログラム: 標準入力から "c3" 形式の座標 (".." でパス) を受け取り、**合法手のみ受理**する `HumanPlayer.java` を実装した。不正入力は再入力させ、パスしか無いときは自動でパスする（リスト 4）。

ソースコード 4: 人間の入力を合法手のみ受理する (`HumanPlayer#think`)

```

1 List<Move> legals = board.findLegalMoves(getColor());
2 if (legals.size() == 1 && legals.get(0).isPass()) return legals.get(0); // パスのみ→自動パス
3 while (true) {
4     Move move = parse(IN.next().trim()); // "c3"→Move, ".."→Pass, 形式不正→null
5     if (move != null && legals.contains(move)) return move; // 合法手のみ受理
6     System.out.println(" 不正な手です。合法手から選んで再入力してください：");
7 }

```

実行例: `MyGame` の片側を `HumanPlayer` に差し替えて対局した。

ソースコード 5: `HumanPlayer` の実行例

```

1 [BLACK] あなたの手番です。着手位置を入力（例：c3）：
2   合法手：[c2, b3, e4, d5]
3   zz ← 不正入力
4   不正な手です。合法手から選んで再入力してください：
5   e4 ← 合法手
6 [BLACK] 合法手なし。パスします。

```

評価・考察: 合法手・不正手・パスのみ・座標範囲外の各ケースで意図通り動作し、成功した。本クラスは AI の強さには影響しない補助クラスだが、自作 AI と人間が直接対局して着手を目視確認でき、評価関数の妥当性を感覚的に検証する土台になった。

課題 1c (200 局の勝率) プログラム・実行例: `Bench c` で `MyPlayer(depth=2)` vs `RandomPlayer` を 200 局（先後各 100）対戦させた。

ソースコード 6: 課題 1c の実行例

```
1 $ java p26x29.Bench c
2 ## 課題 1c: p26x29 vs RandomPlayer 200局 (先手後手各 100)
3 MyPlayer(depth=2) : 177勝 18敗 5分 / 200局 勝率 88.5% 得点率 89.8%
```

評価・考察: 177勝18敗5分 (勝率88.5%, 得点率89.8%) であり, 深さ2の α - β が乱択に圧勝することを当初の見込み通り確認できた. 200局という十分な標本で勝率88.5% (標準誤差約2.3pt) と安定しており, 以降の改良 (1d~1f) はこの値をベースラインとして比較する.

課題 1d (重み行列 M の改良) プログラム: 元の MyEval が外周を一律10としていたのに対し, ImprovedEval は角・X・C・辺・中央を区別した重み行列に細分化した (リスト7). 角は二度と返らない最重要マスなので最大の正 (45), Xマス・Cマスは相手に角を与えやすいので負, 辺は安定しやすいので中程度の正 (11), 中央は早期に占める価値が低いので小さな正 (2), という経験則に基づく.

ソースコード 7: 細分化した重み行列 (ImprovedEval)

```
1 static final float[][] M2 = { // 6x6, 黒視点
2     { 45, -3, 11, 11, -3, 45 }, // 角=45(最重要), C=-3
3     { -3, -7, -4, -4, -7, -3 }, // X=-7(角を渡しやすい)
4     { 11, -4, 2, 2, -4, 11 }, // 辺=11, 中央=2
5     { 11, -4, 2, 2, -4, 11 },
6     { -3, -7, -4, -4, -7, -3 },
7     { 45, -3, 11, 11, -3, 45 } };
```

実行例: 実験プロトコル (改良前→改良後の対 Random 勝率と直接対決) に従い Bench d で測定した.

ソースコード 8: 課題 1d の実行例

```
1 $ java p26x29.Bench d
2 ## 課題 1d: 重み行列M の改良 (depth=2)
3 (1)改良前 vs Random : 92勝 8敗 0分 / 100局 勝率 92.0% 得点率 92.0%
4 (3)改良後 vs Random : 94勝 5敗 1分 / 100局 勝率 94.0% 得点率 94.5%
5 (4)改良後 vs 改良前 : 89勝 6敗 5分 / 100局 勝率 89.0% 得点率 91.5%
```

評価: 結果を表6に示す.

考察: (3)-(1) で対 Random が+2pt, かつ (4) 改良後 vs 改良前が89% (>50%) で大きく勝ち越したため, 改良は当初の狙い通り成功と判定した. 理由は, 角 (45) と X/C マス (負) を明確に区別したことで角の取り合いで優位に立てるためである. 対 Random の差 (+2pt) が小さいのは, 乱択相手では元々勝率が天井近く改善余地が小さいためであり, 改良の効果は同格どうしの直接対決 (4) に最も明確に現れた.

課題 1e (評価関数 ϕ の改良) プログラム: $\phi = w_1\psi + w_2(l_b - l_w) + w_3(n_b - n_w)$ (ψ =マス重み, l =機動力, n =石数) を盤上の石数で序盤・中盤・終盤を判定し段階別重みで実装した (リスト9). 設計の要点は2点である. (1) 位置評価 ψ を主軸 ($w_1 = 1$) に据え,

表 6: 課題 1d: 重み行列の改良 (depth=2, 各 100 局)

対戦	勝/敗/分	勝率
(1) 改良前 vs Random	92/8/0	92.0%
(3) 改良後 vs Random	94/5/1	94.0%
(4) 改良後 vs 改良前	89/6/5	89.0%

機動力は ψ を覆い隠さない範囲 ($w_2 \simeq 1.5$) に抑える. (2) オセロでは中盤に石数が多いのはむしろ不利 (機動力を失う) なので, 石数項 (w_3) を序盤は負・中盤は 0・終盤のみ強い正に効かせ, 取り返されない石差を終盤に積む.

ソースコード 9: 段階別の動的評価 (PhiEval)

```

1 double[] w = (discs <= openMax) ? open : (discs <= midMax) ? mid : end
  ;
2 // w1 w2 w3 段階の境界 openMax=12, midMax=26
3 // open(序盤)= {1.0, 1.5, -0.5} 中盤= {1.0, 1.5, 0.0} end(終盤)=
  {1.0, 0.5, 4.0}
4 return (float)(w[0]*psi + w[1]*(lb-lw) + w[2]*(nb-nw));

```

実行例: Bench e で対 Random と, 1d の改良マス重み (変更前のより強い相手) の両方と対戦させた.

ソースコード 10: 課題 1e の実行例

```

1 $ java p26x29.Bench e
2 ## 課題 1e: 評価関数phi(マス重み+機動力+石数) (depth=2)
3 phi 評価 vs Random : 96勝 2敗 2分 / 100局 勝率 96.0% 得点率 97.0%
4 phi 評価 vs 改良マス重み: 61勝 35敗 4分 / 100局 勝率 61.0% 得点率 63.0%

```

評価: 結果を表 7 に示す.

考察: 対 Random で 96%を保ちつつ, **変更前の改良マス重み (1d) に対しても 61% (35 敗 4 分) と勝ち越し, 変更前より強くなったため改良は成功と判定した** (各 100 局, 400 局でも 61.5%で再現)。ただし一筋縄ではなく, 当初は機動力を $w_2 = 8$ と過大評価したため対改良マス重み 29%まで沈んだ。「機動力+石数を足せば必ず強くなる」わけではなく, depth2 の浅い読みでは過大な機動力項が中盤の評価を歪めて位置評価単体に劣ることを実測した. そこで ψ を主軸に据え機動力を ψ を覆い隠さない範囲へ抑え, **石数項を終盤に限定したところ, 変更前を上回って逆転できた.** 「Random に勝てる」ことと「強い相手に勝てる」ことは別物であり, 後者は段階別の重み分けで初めて達成できるという, 本演習最大の教訓を得た. この段階別評価の発想は大会版へ受け継いだ.

表 7: 課題 1e: ϕ 評価 (depth=2, 各 100 局)

対戦	勝/敗/分	勝率
ϕ vs Random	96/2/2	96.0%
ϕ vs 改良マス重み (変更前)	61/35/4	61.0%

課題 1f (move ordering) プログラム: 各ノードで子手を 1 手だけ進めた局面の評価値 $\phi(q_i)$ で並べ替え, 良い手から先に探索する (max 側は降順, min 側は昇順)。これにより α - β カットが早く起きることを狙う (リスト 11)。枝刈り効果は CountingEval で葉評価の総呼出回数を数えて測った。

ソースコード 11: 1 手読み評価で整列 (OrderingPlayer#ordered)

```
1 List<Move> ordered(Board board, Color color, boolean descending) {
2     List<Move> moves = board.findLegalMoves(color);
3     if (moves.size() <= 1) return moves;
4     Comparator<Move> byPhi = Comparator.comparingDouble(m -> eval.value(
5         board.placed(m)));
6     if (descending) byPhi = byPhi.reversed(); // max 側=降順, min 側=昇順
7     return moves.stream().sorted(byPhi).toList();
8 }
```

実行例: Bench f で depth=4・各 40 局, ランダム順と整列順の評価呼出回数を比較した。

ソースコード 12: 課題 1f の実行例

```
1 $ java p26x29.Bench f
2 ## 課題 1f: move ordering の枝刈り効果 (depth=4, 各 40 局, 評価呼出回数)
3 ランダム順 : 評価呼出 650,207 回, 勝率 97.5%
4 move ordering: 評価呼出 2,238,040 回, 勝率 100.0%
5 評価呼出の削減率 : -244.2%
```

評価: 結果を表 8 に示す。

考察: 勝率は上がった (97.5 → 100%) ものの, 当初の狙いだった評価回数の削減は実現せず約 3.4 倍に**増加し, 本設定では逆効果**と判定した。原因は, 全ノードで全子手を 1 手進めて評価し整列するため, 整列のための浅い評価コストが α - β カットによる削減量を上回ったためである。 $6 \times 6 \cdot \text{depth}4$ は元々探索木が小さく, 「整列の精度 対 計算時間」のトレードオフが時間側に倒れた典型例である。 **成功に向けた方策:** 整列キーをより安価に (置換表のスコアやマス重みのみ) し, ルート近傍の数層だけ整列する, killer-move / 履歴ヒューリスティックで全評価を避ける (詳細は 3.3 節)。

表 8: 課題 1f: move ordering の枝刈り効果 (depth=4, 各 40 局)

手順	value() 呼出回数	勝率
ランダム順	650,207	97.5%
move ordering	2,238,040	100.0%

課題 1g (NegaScout + 置換表付き move ordering) α - β 探索を negamax 形式に統一し, 置換表 (ゼロブリストハッシュ) 付き move ordering と NegaScout (PVS) を実装した (NegaScoutPlayer)。標準初期局面を各深さで 1 回探索し, move ordering のみの α - β を基準に, 置換表・NegaScout を段階的に加えたときの訪問ノード数を計測した (表 9)。NegaScout で全深さにわたり約 **15~26% のノードを削減**しつつ, 三手法の**最善手・評価値は完全に一致**した (探索結果を変えずにコストだけ削減できた)。

表 9: 課題 1g: 枝刈りの段階的強化（標準初期局面・各深さ 1 回・ImprovedEval）

depth	α - β (ordering のみ)	+置換表	+ NegaScout
4	288	288	227 (−21.2%)
5	628	628	490 (−22.0%)
6	1,727	1,717	1,462 (−15.3%)
7	3,392	3,325	2,495 (−26.4%)
8	11,203	10,801	8,652 (−22.8%)

数値は訪問ノード数。括弧内は α - β 比の削減率。

考察: 置換表単体の削減は浅い深さでほぼ 0, 深さ 8 でも 3.6% に留まった。これは強い move ordering の下では同一局面の再訪 (transposition) が少なく, かつ初期局面からの一括探索では合流が起きにくいためである。真価は中盤の合流が多い局面や反復深化との併用で出ると見込む。一方 NegaScout はヌル窓が安定して効き, move ordering との相乗で最も効果が大きかった。1f の「安価な整列キーが必要」という反省どおり, 置換表の最善手を整列の先頭に流用することで, 追加コストなく ordering を強化できた点も設計上の工夫である。

課題 1h(マルチスレッド並列化) ルート分割(root splitting)で並列化した(ParallelPlayer)。ルートの合法手をスレッドへ重複なく割り当てることで, 課題が求める「各スレッドで探索する局面がスレッド間で重複しない工夫」を満たす。各スレッドは専用の置換表を持ちロックなしで部分木を読む。ローカル環境 (10 論理コア)・標準初期局面・depth=9 で計測した (表 10)。

表 10: 課題 1h: ルート分割の速度向上 (depth=9, 標準初期局面)

方式	実時間	総ノード	速度向上
逐次 1 スレッド	4068 ms	15,290	1.00x
(A) フル窓 4 スレッド	2655 ms	33,129	1.53x
(A) フル窓 8 スレッド	2801 ms	33,129	1.45x
(B) α 共有 4 スレッド	4203 ms	19,513	0.97x

(A) 各手をフル窓で読む素のルート分割。(B) PV を先読みして α を確定後, 残りを共有 α で並列化 (YBWC 風)。

考察: 素のルート分割 (A) は 4 スレッドで 1.53 倍に短縮したが, スレッド間で α を共有しないため総ノードが 2.2 倍に膨張した。そこで α 共有 (B) (PV を先に逐次で読んでから残りを並列化) にするとノード膨張は 1.3 倍に抑えられたが, 今度は逐次に読む PV 部分木が全体を支配して (アムダールの法則) 速度向上が消えた。原因は, 6×6 の初期局面はルート合法手が 4 手しかなく, しかも対称で評価が拮抗するため, ルート粒度では並列化できる独立作業が乏しい点にある。実効的な並列化には PV 部分木の内部を再帰的に分割する細粒度並列 (YBWC) が必要だと実測から理解できた。

課題 1i (ビットボード) 6×6 盤を 1 個の long (36 ビット) に詰め、黒石・白石・壁の 3 マスクで表現するビットボード (BitBoard) を実装した。着手可能位置の検出を、マスごとの逐次走査ではなく **8 方向のビットシフトで全マス一括 (並列)** に行う。ランダム対局で集めた 13,009 局面×両色について、配列走査版 findLegalMoves と着手可能マス集合が**完全一致**することを確認し (不一致 0 件)、スループットを比較した (表 11)。

表 11: 課題 1i: 着手可能位置検出のスループット (13,009 局面×2 色×200 反復)

手法	実時間	スループット
配列走査版 findLegalMoves	31,058 ms	0.17 Mcalls/s
ビットボード版 legalMoves	1,213 ms	4.29 Mcalls/s
5,203,600 回の検出を計測。ビットボードが約 25 倍 高速。		

考察: 配列走査版は 36 マス×8 方向をループで走査し、さらに各マスで List を生成するため遅い。ビットボードは全マスをまとめてシフト演算するため分岐とアロケーションが消え、約 25 倍に高速化した。壁 (BLOCK) も「相手石の連結が自然に途切れる」ため変形盤に対応する。これは課題 1k (nps 改善) にも直結する成果である。

課題 1j・1k (その他の改良) 1k (1 秒あたり訪問ノード数の改善) は、本節の置換表 (再探索の削減) とビットボード (合法手検出の約 25 倍高速化) が直接寄与する。1j (その他の方法での評価関数改良) は、大会版の**段階別の動的評価** (「プレイヤについて」節) として実施済みである。

4.2 課題 2

課題 2a (リーグ戦 ap26.league の理解) **プログラム:** 単独でゲームを進める MyGame (課題 1) と異なり、ap26.league.League は複数プレイヤーの総当たり戦を実行する。プログラミング応用杯と同じ枠組みであり、各プレイヤを登録すると全対戦カードを自動で消化し、棋譜 (d2e4c5b3... 形式) と勝敗・石差を出力する。

実行例: 1 局の結果は対戦カード・勝者・石差・棋譜の順に 1 行で出力される。

ソースコード 13: リーグ戦の出力例 (1 対戦カード)

```
1 MY24 vs R -> MY24 won by 12 | d3c3e3f4c5b3...
```

評価・考察: ベースライン (位置評価のみのベースオセロ AI, RandomPlayer) と自作プレイヤを総当たりさせ、勝率・石差・棋譜を機械的に比較できることを確認した。使用法の理解という当初の目的は**達成**し、以降の段階的開発 (2b) の測定基盤として用いた。出力される棋譜を盤面に書き起こせば、負け筋の局面を特定して評価関数の改善につなげられる。

課題 2b (大会用プログラムの段階的開発) 「合法手を返すだけ」→「深さ 2 の α - β 」→「段階別評価関数」→「定石 (book) 導入」と段階的に強くし、各段階でリーグ戦が標準盤・変形盤 (BLOCK) の両方で完走することを確認してから次へ進んだ。開発はグループ

6で git のブランチ運用により分担し、新機能ごとにベースラインとの勝率（各6～100局）で改善を検証した（表2）。段階別評価で p26x42base がベース AI に全勝・平均 26.33 石差、book 導入で p26x42 が p26x42base に 83.33% と、各段階で確実に勝率が改善したことを確認した。

考察: 段階的開発という方針は当初の狙い通り**成功**した。各段階でリーグ戦が標準盤・変形盤の両方で完走することを確認してから次へ進んだため、どの変更が改善・劣化をもたらしたかを勝率で常に追跡でき、最終提出直前の大改造リスクを回避できた。課題 1e で「Random に勝てる＝強い」ではないと学んだ反省を活かし、各段階で必ずベースラインとの直接対決で検証した点が、安定した改善（表2）につながった。

課題 2c（発表 3 要件） 発表資料には必須 3 点を含めた。(1) **探索速度:** cse 本番で標準盤 1 局を計測し book 無効 9.20 Mnps（表3）。高速化の要点は定石 (book) で、総ノード 39%・総思考時間 17.7%を削減した（3.3 節）。(2) **評価関数と勝率改善:** 段階別の動的評価（前半＝機動力・差確定石差・フロンティア差、後半＝機動力最重視）を採用し、ベースラインに対し表2の改善を得た。(3) **その他の工夫:** 定石 (book) による序盤安定、変形盤 (BLOCK) 対応、片側全滅時の石差補正（3.3 節・プレイヤについて節）。チームの独自性は段階別評価と book の組み合わせにある。

考察: 必須 3 点を実測データ付きで発表でき、大会で第 1 位（891 pts, 勝率 88.7%）という当初の目標を上回る結果を得たため、課題 2 は**成功**と結論づける。自分の担当であるテスト・検証では、各段階を多数対局で比較してベース比 100%・平均 26.33 石差、book 版でノード 39%削減を定量的に確認でき、「直感でなく多数対局とプロファイルで検証する」という本演習の教訓を実践できた。

4.3 課題 3

考察: 課題 1～2 を通じ、AI 開発は「思いついた工夫を入れる」段階より「その工夫が本当に効いたかを測る」段階のほうが本質的だと痛感した。実際、機動力を足した初期の ϕ 評価 (1e) は直感では強化のはずが実測では対強相手 30%と失敗し、位置評価を主軸に段階別へ作り直して初めて変更前を上回れた。move ordering(1f) も直感に反し浅い探索では逆効果だった。逆に book は nps を下げつつ総コストを削減するという直感に反する結果だった。したがって今後も、変更ごとにベースラインとの多数対局・プロファイルで定量検証する開発サイクルを徹底したい。

5 参考文献・謝辞

本演習で配布されたリスト 1～9（パッケージ ap26 / p26x29）および総当たり戦フレームワーク ap26.league を参照した。課題 1 の勝率測定は自作の自動対戦ベンチ p26x29.Bench により、課題 2 の探索速度は本番環境 cse.cs.nitech.ac.jp 上での 1 局計測により行った。